

Information Retrieval Overview

This note combines four comprehensive PDF presentations that together explain how to **evaluate** IR systems, work with **benchmark datasets**, understand **web crawling** and ranking algorithms, and explore modern **learning-to-rank** techniques up to large language models.

Content Sources

- Section 1: Provides an in-depth look at **evaluation** and **retrieval models** for IR systems from a 105-page PDF presentation. ([source](#))
- Section 2: Walks through **benchmark datasets** and the **trec_eval** tool for empirical testing from a 119-page PDF presentation. ([source](#))
- Section 3: Covers core concepts of **PageRank** and **SEO** within web search, crawling, and link analysis from a 94-page PDF presentation. ([source](#))
- Section 4: Explores advanced **learning to rank**, **transformers**, and **LLMs** for modern IR from a 169-page PDF presentation. ([source](#))

Section 1: Evaluation of Information Retrieval Systems *This section covers comprehensive evaluation methodologies for IR systems from a 105-slide NPTEL presentation on the "Evaluation" module ([source](#))*

1. Overview & Purpose

Information Retrieval (IR) evaluation is the systematic, empirical assessment of how well a retrieval system satisfies user information needs.

The presentation by **D. Roy (IISER Kolkata)** for the NPTEL *Information Retrieval* course establishes six core objectives:

Goal	Description
Retrieval Models	Introduce keyword matching, probabilistic, language-model, and DFR approaches
Empirical Necessity	Demonstrate why intuition alone is misleading for IR system comparison
Cranfield/TREC Framework	Present benchmark data, relevance judgments, and pooling techniques
Effectiveness Metrics	Define, derive, and exemplify precision, recall, F-measures, MAP, GMAP, MRR, nDCG, etc.
Statistical Comparison	Cover hypothesis testing and inter-annotator agreement

Practical Guidance	Demonstrate TREC-Eval tool usage on real benchmark collections (e.g., FIRE)
--------------------	---

2. Retrieval Models — Foundations

2.1 Keyword-Matching Model

The simplest retrieval approach uses **set-based scoring functions**:

$$\text{Score}(d, Q) = \sum_{t \in Q} f(d, t) \quad \text{or} \quad \prod_{t \in Q} g(d, t)$$

Example: For query "Vehicle Smoke Pollution":

$$\text{Score}(d, Q) = f(d, \text{Vehicle}) + f(d, \text{Smoke}) + f(d, \text{Pollution})$$

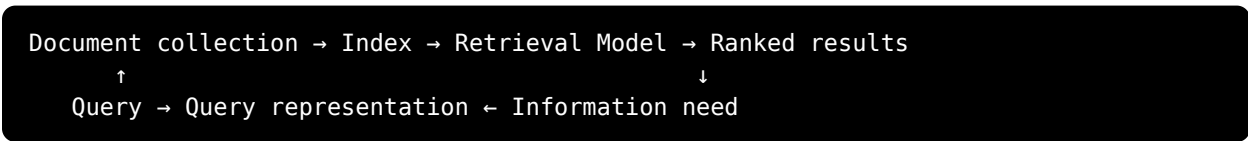
Here, $f(\cdot)$ and $g(\cdot)$ represent term-weighting functions (raw TF, TF-IDF, or probabilistic weights).

2.2 Formal Term-Weighting Models

Model	Weight Formula
Language Model (LM)	$w(q, d)_{\text{LM}} = \frac{\text{tf}(q, d) + \mu \cdot p(q C)}{ d + \mu}$
BM25	$w(q, d)_{\text{BM25}} = \frac{(k_1 + 1) \cdot \text{tf}(q, d)}{\text{tf}(q, d) + k_1 \left(1 - b + b \frac{ d }{\text{avgd}}\right)} \times \log \frac{N - D(q) + 0.5}{D(q) + 0.5}$
DFR	Divergence from Randomness (listed for comparison, not fully expanded)

Key variables: $\text{tf}(q, d)$ = term frequency; $|d|$ = document length; μ = Dirichlet prior; $p(q|C)$ = collection language model; k_1, b = BM25 parameters; avgd = average document length; N = total documents; $D(q)$ = document frequency of q .

2.3 System Architecture



3. Motivation for Comparative Evaluation

IR is an empirical discipline — sophisticated techniques are not automatically superior; validation against baselines is mandatory.

Analogy: "Which is better — Heap sort or Bubble sort?" demands objective, empirical comparison.

Models compared: BM25 (Model 1), Language Model (Model 2), DFR (Model 3)

Criterion	Assessment
Time/efficiency	Computational cost
Quality of ranked list	Effectiveness

Key insight: Rule-based stemming vs. statistical stemming demonstrates that intuition fails — empirical validation is imperative.

4. Benchmarking Framework — Cranfield Method

4.1 Core Components (Cleverdon et al., 1960s)

The Cranfield paradigm provides a reusable, objective test collection with three fixed components: documents, queries, and relevance judgments.

Component	Description
Document collection	Fixed set D of documents
Query/topic collection	Fixed set Q of information needs
Relevance judgments	Binary (relevant/non-relevant) mapping $R \subseteq Q \times D$

Assumptions:

- 1. Relevance can be *objectively* determined
- 2. *All* relevant documents for each query are known
- 3. All relevant documents contribute equally to performance
- 4. Relevance of a document is *independent* of other documents' relevance

Analogy: Syllabus, question paper, and correct answers — analogous to the three components.

4.2 Popular Collections

- **TREC** (Text REtrieval Conference)
- **CLEF** (Cross-Language Evaluation Forum)
- **NTCIR** (NII Test Collection for IR Systems)
- **FIRE** (Forum for Information Retrieval Evaluation)

5. Classical Set-Based Evaluation Metrics

5.1 Confusion Matrix Terminology

	Retrieved	Not retrieved
Relevant	TP (True Positives)	FN (False Negatives)

Non-relevant	FP (False Positives)	TN (True Negatives)
• Perfect system: TP = all relevant, FP = 0, FN = 0, TN = all non-relevant • Useless System A (returns nothing): FN = all relevant • Useless System B (returns everything): FP = all non-relevant		

5.2 Primary Ratios

Recall: $\text{Recall} = \frac{\#\text{TP}}{\#\text{TP} + \#\text{FN}} = \frac{\text{relevant retrieved}}{\text{total relevant}}$

Precision: $\text{Precision} = \frac{\#\text{TP}}{\#\text{TP} + \#\text{FP}} = \frac{\text{relevant retrieved}}{\text{total retrieved}}$

F-measure (harmonic mean): $F_1 = \frac{2PR}{P+R}$

Generalized F-measure (β -weighted): $F_\beta = \frac{(1+\beta^2)PR}{\beta^2P+R}$

- $\beta > 1 \rightarrow$ recall emphasized
- $\beta < 1 \rightarrow$ precision emphasized
- $\beta = 1 \rightarrow F_1$

5.3 Trade-off Illustration

Precision-recall trade-off: Increasing recall (retrieving more documents) typically decreases precision (more non-relevant items included).

Visual precision-recall curves demonstrate that **same set-based P/R/F does not capture ranking quality** — two systems with identical F-scores can produce dramatically different user experiences.

5.4 Accuracy — Why It Fails in IR

Scenario	Accuracy	Reality
Typical collection	~1M documents, ~50 relevant per query	Class imbalance extreme
Useless System A (returns nothing)	~99.995%	Misleadingly high
Useless System B (returns everything)	~0.005%	Misleadingly low

Conclusion: Accuracy is **meaningless** for IR; metrics must focus on the *rare* relevant class.

6. Ranking-Sensitive Metrics

6.1 Precision@K (P@K)

Precision@K: Precision computed on the *top-K* retrieved documents, where K is a fixed cut-off.

- **K** = fixed cut-off (e.g., 5, 10)
- **Usage**: Web search — users typically examine only first K results
- **Critical note**: *K* must be **identical for all queries** when comparing systems

6.2 R-Precision

R-Precision: Precision at rank R, where R equals the number of relevant documents for that specific query.

- **R** = number of relevant documents (varies per query)
- **Interpretation**: Measures how many of the *known relevant* documents appear within the top *R* positions

6.3 R-Precision Example

System	Ranking (R = 8)	Relevant docs in top 8	R-Precision
A	R, N, R, R, N, N, R, N, ...	4	0.5
B	R, R, R, R, R, R, R, R, ...	8	1.0

6.4 Average Precision (AP)

Average Precision (AP): The average of precision values **at each rank where a new relevant document is retrieved**.

$$AP(Q) = \frac{1}{N_{rel}} \sum_{i \in Rel} \frac{i}{rank(d_i)}$$

Where N_{rel} = number of relevant documents for query *Q*.

Worked Example (5 relevant docs):

Rank	Type	Recall	Precision
1	R	0.2	1.00
2	N	0.2	0.67
3	R	0.4	0.67
4	R	0.6	0.75
5-6	N	0.6	0.50
7	R	0.8	0.57
10	R	1.0	0.50

$$AP = \frac{1}{5} \left(1 + \frac{2}{3} + \frac{3}{4} + \frac{4}{7} + \frac{5}{10} \right) = 0.76$$

Properties:

- Emphasizes **early** retrieval of relevant documents
- Handles **binary relevance** only (0/1)

6.5 Mean Average Precision (MAP)

MAP: The mean of AP values across multiple queries.

$$MAP = \frac{1}{|Q|} \sum_{q \in Q} AP(q)$$

Example (3 queries):

Query	AP
Q ₁ (4 rel)	0.76
Q ₂ (3 rel)	0.207
Q ₃ (7 rel)	0.182

$$MAP = \frac{0.76 + 0.207 + 0.182}{3} = 0.383$$

6.6 11-Point Interpolated Average Precision

For each standard recall level $r_i \in \{0.0, 0.1, \dots, 1.0\}$:

$$P_{\text{int}}(r_i) = \max_{r' \geq r_i} P(r')$$

This **forces a monotonic decreasing precision curve** (no dips), enabling standardized comparison across systems.

6.7 Geometric MAP (GMAP)

GMAP: Penalizes systems that perform very badly on any query by using geometric rather than arithmetic mean.

$$GMAP = \exp \left(\frac{1}{|Q|} \sum_{q \in Q} \ln(AP(q)) \right)$$

Systems with near-zero AP on any query suffer severe GMAP penalties, unlike MAP which allows one bad query to be averaged out.

7. Statistical Comparison of Systems

7.1 Hypothesis Testing

Aspect	Description
Null hypothesis (H₀)	No significant difference between systems

Alternative (H_1)	Significant difference exists
Paired t-test	Compare MAP scores across same queries
Wilcoxon signed-rank	Non-parametric alternative for small samples
Sign test	Simple non-parametric test

7.2 Inter-Annotator Agreement

Measure	Use Case
Cohen's Kappa (κ)	Two annotators, categorical labels
Fleiss' Kappa	Multiple annotators, same items
Krippendorff's Alpha	Any number of annotators, any scale

Interpretation: $\kappa > 0.8$ excellent; 0.6–0.8 good; 0.4–0.6 moderate; < 0.4 poor.

8. Practical TREC-Eval Usage

8.1 File Formats

File Type	Format	Example
Run file	query-id Q0 doc-id rank score RUN-ID	126 Q0 en.2.281.34.2009.9.11 1 14.693296432495117 BM25
Qrels file	query-id 0 doc-id relevance	126 Q0 en.13.10.19.2009.7.30 1

8.2 Command-Line Syntax

```
# All default metrics
trec_eval qrels.txt my_run.txt

# Specific metrics
trec_eval -m map -m P.10 -m ndcg -m Rprec qrels.txt my_run.txt

# Per-query scores
trec_eval -q qrels.txt my_run.txt

# All TREC metrics
trec_eval -m all_trec qrels.txt my_run.txt
```

8.3 Metric Catalogue

Category	Metric	Command	Description
----------	--------	---------	-------------

Basic	MAP	-m map	Mean Average Precision
	GMAP	-m gm_map	Geometric MAP
	P@K	-m P.10	Precision at rank K
	nDCG	-m ndcg	Normalized Discounted Cumulative Gain
	nDCG@K	-m ndcg_cut.10	nDCG at cut-off K
	R-Precision	-m Rprec	Precision at R (number of relevant docs)
	MRR	-m recip_rank	Mean Reciprocal Rank
Recall-oriented	Recall@K	-m recall.1000	Recall after K documents
	Interpolated P@R	-m iprec_at_rec.0.5	Precision at 50% recall
Advanced	bpref	-m bpref	Binary Preference (incomplete judgments)
	infAP	-m infAP	Inferred Average Precision
	instrec	-m instrec	Instant Recall (early precision)

8.4 Interpreting Results — Red Flags

Symptom	Interpretation
MAP < 0.1	System malfunctioning or severely under-performing
P@10 = 0	No relevant documents in top-10
num_rel_ret = 0	Complete retrieval failure
High variance across queries	Inconsistent behavior; needs query-specific tuning

8.5 Comparative Example

System	MAP	P@10	nDCG
BM25	0.2987	0.4220	0.6278
LM-Dir	0.3129	0.4660	0.6486

Finding: LM-Dirichlet improves over BM25 on all three metrics.

Section 2: Indexing, Retrieval, and Relevance Feedback in Benchmark Datasets

This section covers practical benchmark evaluation and relevance feedback techniques from a 119-slide NPTEL presentation ([source](#))

1. Benchmark IR Forums and Collections

1.1 Major Evaluation Forums

Forum	Full Name	Region/Focus	Core URL
TREC	Text REtrieval Conference	U.S.	https://trec.nist.gov/
CLEF	Cross-Language Evaluation Forum	European, multilingual	https://www.clef-initiative.eu/
NTCIR	NII Test Collection for IR Systems	Asian, NPTEL-associated	https://research.nii.ac.jp/
FIRE	Forum for Information Retrieval Evaluation	Indian IR Society	https://fire.irs1.org.in/

FIRE specific resources:

- Data: <https://fire.irs1.org.in/static/data>
- Organizational form: <https://fire.irs1.org.in/fire/download/organisational.pdf>
- Individual form: <https://fire.irs1.org.in/fire/download/individual.pdf>

1.2 Three-Component Structure

All Cranfield-style collections share:

Document Collection	← The corpus (fixed)
Query Collection	← Information needs (fixed)
Relevance Judgments	← Binary/graded labels (qrels)

2. The trec_eval Evaluation Toolkit

2.1 Problem Statement

Given: Ranked result lists from retrieval systems + relevance judgment files (qrels) **Goal:** Compute IR effectiveness metrics automatically and consistently

2.2 Solution Overview

trec_eval is the **official open-source evaluation software** used by TREC since 1992:

Attribute	Details
Type	Command-line tool
Metrics	20+ standard IR metrics
Scale	Handles large-scale runs
GitHub	https://github.com/usnistgov/trec_eval
Conda	<code>conda install -c conda-forge trec_eval</code>
HuggingFace	https://huggingface.co/spaces/evaluate-metric/trec_eval

2.3 File Formats

Run File (System Output)

Column	Meaning	Example
1. query-id	Topic identifier	126
2. Q0	Literal "Q0" (ignored)	Q0
3. doc-id	Document identifier	en.2.281.34.2009.9.11
4. rank	Integer rank (1-based)	1
5. score	Retrieval score (float)	14.693296432495117
6. RUN-ID	System/run identifier	BM25

Example line: 126 Q0 en.2.281.34.2009.9.11 1 14.693296432495117 BM25

Qrels File (Ground Truth)

Column	Meaning	Example
1. query-id	Topic identifier	126
2. 0	Literal "0" (ignored)	Q0
3. doc-id	Document identifier	en.13.10.19.2009.7.30
4. relevance	Binary: 0=non-relevant, 1=relevant	1

Example line: 126 Q0 en.13.10.19.2009.7.30 1

2.4 Command-Line Syntax

```
# Basic syntax
trec_eval [options] qrels_file results_file

# All default metrics
trec_eval qrels.txt my_run.txt

# Specific metrics
trec_eval -m map -m P.10 -m ndcg -m Rprec qrels.txt my_run.txt

# Per-query scores
trec_eval -q qrels.txt my_run.txt

# All TREC metrics
trec_eval -m all_trec qrels.txt my_run.txt
```

2.5 Common Options

Option	Meaning
-m metric	Compute only listed metric (repeatable)
-q	Output per-query results (one line per query)
-m all_trec	Produce full list of supported metrics

2.6 Sample Output Interpretation

```
runid      all    LMDIR
num_q      all    50
num_ret    all    50000
num_rel_ret all    2761
map        all    0.3129
gm_map     all    0.2210
Rprec      all    0.3247
recip_rank all    0.6378
P_5        all    0.4960
P_10       all    0.4666
```

Field	Meaning
num_q	Number of queries evaluated
num_ret	Total retrieved documents across all queries
num_rel_ret	Total <i>relevant</i> documents retrieved
map	Mean Average Precision

gm_map	Geometric MAP
Rprec	R-Precision
recip_rank	Mean Reciprocal Rank
P_k	Precision at rank k

2.7 Metric Catalogue

Category	Metric	Command	Description
Basic	MAP	-m map	Mean Average Precision
	GMAP	-m gm_map	Geometric MAP
	P@K	-m P.10, -m P.20	Precision at K
	nDCG	-m ndcg	Normalized DCG (full list)
	nDCG@K	-m ndcg_cut.10	nDCG at cut-off K
	R-Precision	-m Rprec	Precision at R (#relevant docs)
	MRR	-m recip_rank	Mean Reciprocal Rank
Recall-oriented	Recall@K	-m recall.1000	Recall after K docs
	Interpolated P@R	-m iprec_at_rec.0.5	Precision at 50% recall
Advanced	bpref	-m bpref	Binary Preference (incomplete judgments)
	infAP	-m infAP	Inferred Average Precision
	instrec	-m instrec	Instant Recall (early precision)

2.8 FIRE Dataset Practical Example

Run file format (fire_lmdir_run.res):

```
126 Q0 en.2.281.34.2009.9.11 1 17.589683532714944 LMDIR
126 Q0 en.3.329.19.2009.9.22 2 17.534711837768555 LMDIR
126 Q0 en.3.333.11.2009.10.24 3 17.205379486083984 LMDIR
...
```

Evaluation commands:

```
# Specific metrics
trec_eval -m map -m P.10 -m ndcg -m Rprec en.qrels.126-175.2011.txt fire_lmdir_run.res

# All metrics
trec_eval -m all_trec en.qrels.126-175.2011.txt fire_lmdir_run.res

# Per-query analysis
trec_eval -m all_trec -q en.qrels.126-175.2011.txt fire_lmdir_run.res > results.txt
```

2.9 Results Interpretation — Red Flags

Symptom	Interpretation
MAP < 0.1	System malfunctioning or severely under-performing
P@10 = 0	No relevant documents in top-10
num_rel_ret = 0	Complete retrieval failure
Very high variance across queries	Inconsistent behavior; needs query-specific tuning

Comparative example:

System	MAP	P@10	nDCG
BM25	0.2987	0.4220	0.6278
LM-Dir	0.3129	0.4660	0.6486

Finding: LM-Dirichlet outperforms BM25 across all metrics.

Section 3: Web Search, Crawling, PageRank, HITS, and Search Engine Optimization

This section covers web-scale IR architecture, link analysis algorithms, and SEO from a 94-slide NPTEL presentation ([source](#))

1. Course Roadmap & Context

Completed	Current & Upcoming
Bag-of-Words models (VSM, BM25, Language Models)	Web Search & Crawling
Relevance Feedback & Vocabulary-mismatch solutions	PageRank Algorithm

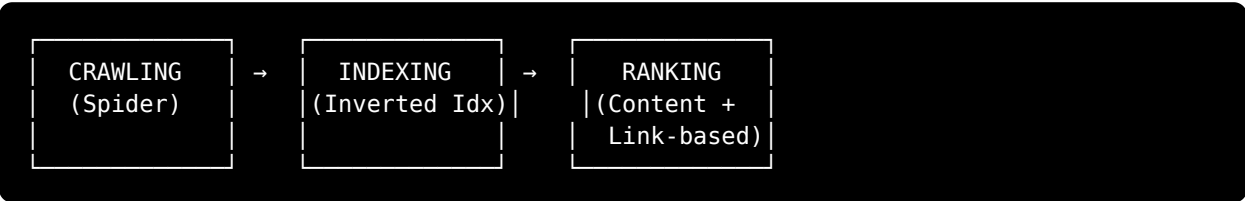
• Evaluation metrics	• HITS Algorithm
• Introduction to Embeddings & Transformers	• Search-Engine Optimization (SEO)

2. The Web as a Corpus — Key Differences

Feature	Traditional IR	Web Search
Collection size	Thousands–millions	Billions–trillions
Structure	Controlled, curated	Completely uncontrolled
Quality	High, consistent	Highly variable
Dynamism	Static	Constantly changing
Links	Few / normalized	Rich, complex graph structure
Spam	Minimal	Pervasive and sophisticated

Key insight: Scale, heterogeneity, dynamism, link richness, and spam prevalence demand fundamentally different architectures and algorithms.

3. Web-Search Architecture



4. The Web Crawler (Spider/Robot)

4.1 Definition

Web crawler: An automated program that systematically browses the WWW to build an index.

4.2 Use Cases

- News aggregation
- Technical documentation archiving
- Academic paper indexing (e.g., DBLP)
- Product/movie review collection
- Social-media monitoring

4.3 Basic Crawler Algorithm

Step	Action
①	Initialize frontier with seed URLs
②	Dequeue URL from frontier
③	Fetch page (HTTP GET)
④	Store downloaded content
⑤	Extract hyperlinks from page
⑥	Add extracted links to frontier (after filtering)
⑦	Repeat ②–⑥ until termination

4.4 Frontier Management

Aspect	Technique
Priority Queue	Determines crawl order by domain importance, freshness, PageRank
Politeness	Obey robots.txt, respect Crawl-delay, limit per-site rate
Freshness	Schedule revisits based on observed change frequency

4.5 Core Components

Component	Role	Key Challenge
DNS Resolver	Hostname → IP conversion	Major bottleneck → requires aggressive caching
Parser	Extract text & links	Handle diverse markup formats (HTML, PDF, etc.)
Fetching Engine	HTTP client	Follow redirects, handle errors, manage connections
URL Filter	Duplicate detection, domain restrictions	Content-type filtering, URL normalization

5. Major Crawling Challenges

5.1 Robustness

- HTML syntax errors and malformed documents

- **Spider traps**: Sites generating infinite URL spaces (e.g., calendar navigation with unbounded dates)

5.2 Content Selection

Challenge	Consideration
Content type	News, reviews, academic, etc.
Content quality	High-quality vs. malicious/adversarial
Deduplication	Exact and near-duplicate detection

5.3 Scheduling Trade-offs

Trade-off	Description
Coverage vs. Freshness	High-value or rapidly changing pages need priority fetching and frequent revisits
Exploration vs. Exploitation	Balance discovering new sites vs. deepening known high-quality sites

5.4 Performance & Scalability

- Compute, storage, bandwidth requirements
- **Distributed architecture** for horizontal scaling
- **Fault tolerance** and easy machine addition

5.5 Politeness & Legal/Ethical Issues

- Respect per-site rate limits; avoid denial-of-service
- Honor **robots.txt** and **Crawl-delay** directives
- Observe copyright and terms of service

6. Spam & Problematic Content

Issue	Description	Remedy
Crawler traps	Auto-generated infinite URL spaces	BEAST (Budget Enforcement with Anti-Spam Tactics): assign budget per site, prioritize by remaining budget/reputation
Keyword stuffing	Insert high-search-volume terms	Content quality signals, spam detection classifiers
Link spam	Create cliques to manipulate PageRank	Link-based spam detection, TrustRank
Cloaking	Different content to crawlers vs. humans	JavaScript execution in sandboxed engines, multi-agent

		verification
--	--	--------------

7. Near-Duplicate Detection

7.1 Shingling (Character-level)

k-shingle: Contiguous substring of length k extracted from a document.

Example (document: "the cat sat"):

k	Shingles
2	"th", "he", "e ", " c", "ca", "at", "t ", " s", "sa"
3	"the", "he ", "e c", "cat", "at ", "t s", "sat"

Choosing k:

- Small $k \rightarrow$ many matches, low discriminative power
- Large $k \rightarrow$ fewer matches, high discriminative power
- **Typical**: $k = 5-10$ for English text

7.2 Scaling Pairwise Comparisons

Direct $O(n^2)$ pairwise Jaccard similarity is infeasible for web scale.

Solution: Min-hashing + Locality-Sensitive Hashing (LSH)

Step	Technique	Purpose
1	Minhashing	Compact signature preserving Jaccard similarity
2	LSH	Hash signatures into buckets; only compare documents in same bucket

This dramatically reduces comparisons from $O(n^2)$ to near-linear.

8. Distributed Indexing

Technique	Purpose
Sharding	Split inverted index across machines
MapReduce	Parallel processing: document parsing, tokenization, posting-list construction

The **inverted index** remains core, now stored in distributed, sparse format.

9. PageRank Algorithm

9.1 Motivation — "Web Search Quality Problem"

Classical IR models (VSM, BM25) ignore **page quality/authority**. On the open web, spam pages can be highly relevant; an **independent authority measure** is essential.

9.2 Link-Analysis Intuition (Citation Analogy)

Domain	Mechanism	Signal
Academia	Citations	Paper importance
Web	In-links	Page importance

Key insight (Brin & Page, 1998): "A page is important if it is pointed to by other important pages."

9.3 Basic PageRank Formula

$$PR(A) = \sum_{i=1}^n \frac{PR(p_i)}{c(p_i)}$$

Where:

- $PR(A)$ = PageRank of page A
- p_i = pages that link to A
- $c(p_i)$ = number of outbound links from p_i

Initialization: $PR^{(0)}(A) = 1/N$ where N = total pages

Computation: Iterative until convergence (power method)

9.4 Matrix Representation

- **Transition matrix H** (column-stochastic): each column j contains $1/c(j)$ for outgoing links, 0 otherwise

$$\mathbf{PR}^{(t+1)} = H, \mathbf{PR}^{(t)}$$

Convergence: When $H\mathbf{x} = \mathbf{x}$ (principal eigenvector, eigenvalue = 1)

9.5 Random Surfer Model & Damping

Random surfer: With probability d , follows an outgoing link; with probability $(1-d)$, jumps to random page.

Damping factor d (typically **0.85**):

- Ensures ergodicity
- Avoids sink traps
- Accelerates convergence

$$PR = (1 - d) \cdot I + d \cdot H$$

Or element-wise:

$$PR_{t+1}(A) = \frac{1-d}{N} + d \sum_{i=1}^n \frac{PR_t(p_i)}{c(p_i)}$$

9.6 Handling Special Cases

Situation	Remedy
Dangling nodes (no out-links)	Distribute rank uniformly to all pages (modify H)
Spider traps (closed sub-graph)	Damping factor's random jumps break the trap
Disconnected components	Damping factor connects all nodes via teleportation

9.7 Scalability

Technique	Purpose
Sparse matrix storage	Only non-zero entries (outgoing links)
Distributed computation (MapReduce)	Parallel PageRank computation

MapReduce PageRank:

- *Mapper*: Emit contribution of each page to its out-neighbors
- *Reducer*: Sum contributions, apply damping, produce new rank vector

9.8 Integration with Traditional IR

Final ranking combines **content score** (BM25, LM, term features) with **authority score** (PageRank):

$$\text{FinalScore} = \alpha \cdot \text{ContentScore} + \beta \cdot \text{PageRank}$$

PageRank provides a **query-independent quality signal** complementing query-dependent content relevance.

9.9 Limitations & Criticisms

Issue	Description
Rich-get-richer	Bias toward older, established pages
Link-spam vulnerability	Susceptible to link farms
Query-independent	Not topic-specific
Computational cost	Heavy computation; freshness lag

10. HITS Algorithm (Hyperlink-Induced Topic Search)

10.1 Contrast with PageRank

Aspect	PageRank	HITS
Computation	Pre-computed, global	Query-time, local (base set)
Scores	Single global importance	Two scores : Hub and Authority
Scope	Entire web graph	Focused sub-graph (root + base set)
Damping	Yes (0.85)	No (original formulation)
Stability	Very stable	Sensitive to root-set changes
Use case	General importance	Topic-specific communities

10.2 Core Intuition — Mutual Reinforcement

Good Hub: Points to many **good authorities** on a topic **Good Authority**: Is pointed to by many **good hubs**

10.3 Algorithmic Steps (Query-Driven)

Step	Action
1	Query processing — user submits query
2	Root set — retrieve top- t results (e.g., $t = 200$) using text-based IR
3	Base set expansion — add pages linking to or from root set (1,000–5,000 pages)
4	Link graph construction — build adjacency matrix A for base set
5	Iterative computation — compute hub and authority scores
6	Ranking — return pages sorted by authority (optionally by hub)

10.4 Mathematical Formulation

Let A be the adjacency matrix where $A_{ij} = 1$ iff page i links to page j .

Authority update: $\text{auth}^{(k)} = A^\top \text{hub}^{(k-1)}$

Hub update: $\text{hub}^{(k)} = A \text{auth}^{(k-1)}$

Normalization: After each iteration, divide each vector by its **L1 norm**.

Eigen-interpretation:

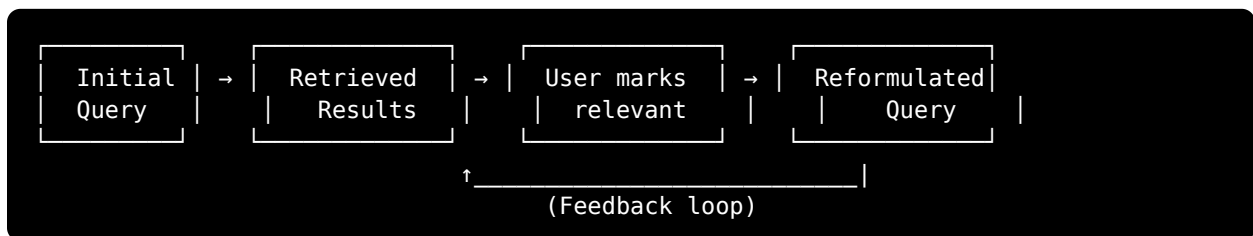
- Authority vector $\rightarrow$ principal eigenvector of $A^T A$
- Hub vector $\rightarrow$ principal eigenvector of $A A^T$

11. Relevance Feedback (RF) — Theory & Practice

11.1 Motivation

Problem	Description
Vocabulary mismatch	Users and authors use different terms for same concepts $\rightarrow$ reduced recall
User uncertainty	Users don't know exact terms but recognize relevance when shown
Goal	Build better representation of information need via query expansion/re-weighting

11.2 High-Level RF Workflow



11.3 Types of Relevance Feedback

Type	Description	Data Source
Explicit RF	User actively marks documents as relevant/non-relevant	Direct user input
Implicit RF	System infers relevance from user behavior	Clicks, dwell time, scroll depth, reformulations
Blind/Pseudo RF	System assumes top-k results are relevant	No user input; automatic top-k feedback

Section 4: Learning to Rank, Semantic Indexing, and Neural IR

This section covers the evolution from traditional IR to neural approaches including LSI, embeddings, Transformers, and LLMs from a 169-slide NPTEL presentation ([source](#))

1. Introduction & Historical Evolution

Era	Core Question	Key Innovation
Traditional IR	Hand-crafted scoring for every query/domain?	VSM, BM25, Language Models – fixed formulas
Learning to Rank	Can we <i>learn</i> the optimal ranking function?	Data-driven, adaptable ranking models
Latent Semantic Indexing	How to overcome vocabulary mismatch?	Low-rank approximation capturing latent concepts
Neural Embeddings	How to represent semantics densely?	Word2Vec, GloVe, sentence/document embeddings
Transformers/BERT	How to get contextual, query-aware representations?	Deep bidirectional understanding, re-ranking
Large Language Models	Can generative models become the core of IR?	RAG, LLM-as-judge, autonomous retrieval agents

2. The Ranking Problem — Formalization

Input: Query q and candidate set $D = \{d_1, ..., d_n\}$

Goal: Learn scoring function $f(q,d) \rightarrow \mathbb{R}$

Output: Documents ordered descending by $f(q,d)$

Supervised setting: Labeled triples (q_i, d_i, y_i) where $y_i \in \{\text{relevance grades}\}$

3. Learning to Rank (LTR)

3.1 Definition

Learning to Rank: A machine-learning framework that automatically builds a ranking model from labeled query-document pairs.

Element	Description
Input	Feature vector $\phi(q,d) \in \mathbb{R}^m$
Output	Ranked list for each query
Training data	Human-annotated relevance judgments

3.2 Formalism

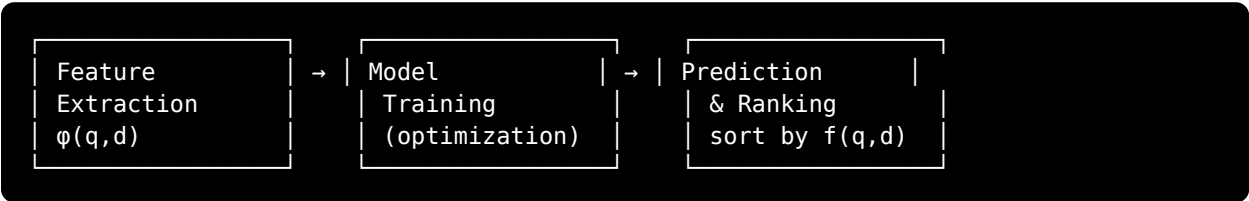
- Query set $\mathbf{Q} = \{q_1, ..., q_m\}$

- For each q : document set $D_q = \{d_1, \dots, d_n\}$
- Feature vector: $\phi(q,d)$
- Relevance label: $y(q,d) \in Y$ (ordinal or continuous)

Learning objective: Find $f: \mathbb{R}^m \rightarrow \mathbb{R}$ such that:

$$f(\phi(q,d_i)) > f(\phi(q,d_j)) \iff d_i \text{ is ranked higher than } d_j$$

3.3 Supervised LTR Pipeline



3.4 Feature Design

Category	Typical Features
Query-dependent	BM25 score, LM score, query term frequency, document length
Query-independent (static)	PageRank, URL depth, domain authority
Interaction	Embedding similarity, term proximity, click-through rate
Cross-modal	Visual features, audio embeddings

Best practices: Normalization, missing-value imputation, feature selection, cross-validation, domain adaptation.

3.5 LTR Approaches — Three Paradigms

Approach	View	Loss	Algorithms	Complexity	IR-Metric Alignment
Pointwise	Regression/classification per (q,d)	MSE, cross-entropy	Linear regression, regression trees	Low	Poor — ignores relative order
Pairwise	Binary classification of document pairs	Pairwise logistic/hinge	RankNet, RankBoost, LambdaRank	Medium	Good — optimizes relative order

Listwise	Direct optimization of entire ranked list	Listwise likelihood, NDCG surrogates	ListNet, ListMLE, LambdaMART	High	Excellent – optimizes list-wise metrics
-----------------	---	--------------------------------------	------------------------------	------	--

3.5.1 Pointwise Formulation

Minimize: $L = \sum_i h(f(x_i), y_i)$

Where h is MSE for regression or cross-entropy for classification.

3.5.2 Pairwise Formulation

Minimize: $L = \sum_{i,j} \ell(f(x_i, x_j), \text{sign}(y_i - y_j))$

RankNet:

- Neural net computes scores $s_i = f(x_i)$
- Pairwise probability: $P_{ij} = \frac{1}{1 + e^{-\sigma(s_i - s_j)}}$
- Loss = cross-entropy between P_{ij} and true preference

LambdaRank / LambdaMART:

- Extends RankNet: gradient multiplied by **ΔNDCG** from swapping pair
- **LambdaMART**: Combines LambdaRank gradients with **Gradient-Boosted Decision Trees (GBDT)**
- **De-facto state-of-the-art** for web search (Yahoo! LTR Challenge winner)

3.5.3 Listwise Formulation

Minimize: $L = \sum_q I(\pi_f(q), \pi^*(q))$

ListNet:

- Uses **Plackett-Luce** top-one probability: $P(d_i) = \frac{e^{s_i}}{\sum_j e^{s_j}}$
- Cross-entropy between predicted and ground-truth top-one distributions

ListMLE:

- Maximizes likelihood of entire ground-truth permutation under Plackett-Luce
- Computationally cheaper than ListNet

3.6 Neural Learning-to-Rank

Sub-type	Core Idea	Examples
Representation-focused	Learn better query/document embeddings	BERT, Siamese networks
Interaction-focused	Model fine-grained token-level interactions	Duet, ColBERT
End-to-End	Jointly learn representation + ranking loss	Deep RankNet

3.7 Contextual & Personalized Ranking

Signal Type	Examples
Context-aware	User location, device, time-of-day
Personalization	User history, preferences, demographics
Multi-task learning	Optimize relevance, diversity, fairness simultaneously

Emerging directions: Fairness & bias mitigation, explainability

3.8 Evaluation Metrics Summary

Metric	Formula/Description
Precision@k	Fraction of relevant docs in top-k
Recall@k	Fraction of all relevant docs in top-k
MAP	Mean of Average Precision across queries
MRR	Mean reciprocal rank (1/rank of first relevant)
NDCG@k	$DCG@k = \sum_{i=1}^k \frac{2^{rel_i} - 1}{\log_2(i+1)}$; $NDCG@k = \frac{DCG@k}{IDCG@k}$

Metric-aware optimization: LambdaRank scales gradients by change in NDCG from swapping documents, enabling surrogate loss to target non-smooth metrics.

4. Latent Semantic Indexing (LSI) 🔍

4.1 Linear Algebra Foundations

Concept	Definition
Matrix Rank	Number of linearly independent rows/columns ($\leq \min\{m,n\}$)
Diagonal Matrix	Square with non-zeros only on diagonal; rank = #non-zero diagonals
Eigenvalues/vectors	For square C , λ is eigenvalue if $C \cdot x = \lambda x$; x is eigenvector

Finding eigenvalues: Solve $\det(C - \lambda I) = 0 \rightarrow$ polynomial of degree $m \rightarrow$ at most m roots.

4.2 Vector-Space Similarity (Baseline)

$$\text{score}(q, d) = \frac{\sum_i q_i d_i}{\sqrt{\sum_i q_i^2} \sqrt{\sum_i d_i^2}}$$

Limitation: Exact term matching; suffers from **vocabulary mismatch** (synonymy, polysemy, noise).

4.3 Vocabulary-Mismatch Illustration

Term Set	Doc 1 (vehicle-related)	Doc 2 (smoke-related)
Vehicle/Smoke/Pollution	"Passenger vehicles ... air pollution ... smoke"	"Exhaust gas ... toxic ... pollution"

Both discuss *pollution* but term overlap is low → poor recall with exact matching.

4.4 LSI Motivation

Goal: Capture latent "concepts" linking terms and documents, enabling retrieval even without exact term matches.

Mathematical objective: Find low-rank approximation A' of term-document matrix A (size $m \times n$) minimizing Frobenius norm:

$$\min_{A', \text{rank}(A')=k} \|A - A'\|_F$$

Where: $\|X\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n X_{ij}^2}$

4.5 Singular Value Decomposition (SVD)

For any A (rank r), there exist orthogonal matrices U ($m \times r$), V ($r \times n$) and diagonal S ($r \times r$) such that:

$$A = U, S, V^\top$$

Truncated SVD for LSI (rank $k < r$):

$$A_k = U_k, S_k, V_k^\top$$

Where U_k, S_k, V_k keep only top k singular values/vectors.

Query processing in LSI space: $\tilde{q} = q^\top U_k S_k^{-1}$

Then rank documents by similarity in reduced k -dimensional space.

Section 4: Neural IR — From Embeddings to Large Language Models

This section covers the complete evolution from LSI through neural embeddings, Transformers, and LLMs for IR from a 169-slide NPTEL presentation ([source](#))

1. Introduction & Scope

Topic	Core Question	Evolutionary Significance
Traditional IR	Hand-crafted scoring for every scenario?	Fixed formulas: VSM, BM25, LM
Learning to Rank	Can we <i>learn</i> optimal ranking?	Data-driven adaptation
LSI	Overcome vocabulary mismatch?	Latent concept capture
Neural Embeddings	Dense semantic representations?	Direct synonymy/polysemy handling
Transformers/BERT	Contextual, query-aware representations?	Deep bidirectional understanding
Large Language Models	Generative models as IR core?	RAG, LLM-as-judge, autonomous agents

2. Learning to Rank – Detailed Treatment

2.1 The Ranking Problem

Input: Query q , candidate documents $D = \{d_1, ..., d_n\}$

Goal: Learn $f(q,d) \rightarrow \mathbb{R}$ producing optimal document ordering

Supervised setting: Labeled triples (q_i, d_i, y_i) with relevance grades

2.2 Three LTR Paradigms (Detailed)

Approach	Mathematical View	Key Algorithms	IR-Metric Alignment
Pointwise	Regression/classification: $f(x_i) \approx y_i$	Linear regression, regression trees, neural nets	Poor — ignores relative ordering
Pairwise	Classification: $f(x_i) > f(x_j) \Leftrightarrow y_i > y_j$	RankNet, RankBoost, LambdaRank	Good — optimizes pairwise preferences
Listwise	Permutation optimization: $\pi_{\cdot} f \approx \pi^*$	ListNet, ListMLE, LambdaMART	Excellent — optimizes list-level metrics

Pointwise Details

Minimize: $L = \sum_i h(f(x_i), y_i)$

Where h is MSE for regression or cross-entropy for classification.

Critical flaw: A document with relevance 3 vs. 4 is treated like 0 vs. 1 — the *relative* ordering that matters for ranking is lost.

Pairwise Details — RankNet

Score computation: $s_i = f(x_i)$ (neural network)

Pairwise probability via logistic function: $P_{ij} = \frac{1}{1+e^{-\sigma(s_i-s_j)}}$

Loss: Cross-entropy between P_{ij} and true preference (0 or 1)

LambdaRank Enhancement:

 Multiply gradients by **ΔNDCG** — the change in NDCG from swapping documents i and j

This directly optimizes a non-differentiable metric through a differentiable surrogate.

LambdaMART:

Component	Description
Base learner	Gradient-Boosted Decision Trees (GBDT)
Gradient source	LambdaRank gradients (NDCG-weighted)
Status	De facto state-of-the-art for web search (Yahoo! LTR Challenge winner)

Listwise Details

ListNet:

Uses **Plackett-Luce** top-one probability: $P(d_i) = \frac{e^{s_i}}{\sum_j e^{s_j}}$

Loss: Cross-entropy between predicted and ground-truth top-one distributions

ListMLE:

Maximizes likelihood of **entire ground-truth permutation** under Plackett-Luce — computationally cheaper than ListNet.

2.3 Neural LTR Sub-Types

Sub-type	Core Idea	Examples
Representation-focused	Learn better embeddings	BERT, Siamese networks, DSSM
Interaction-focused	Model token-level interactions	Duet, ColBERT, KNRM
End-to-end	Joint representation + ranking	Deep RankNet, SetRank

2.4 Contextual & Personalized Ranking

Signal Type	Examples
Context-aware	Location, device, time-of-day, task context
Personalization	Search history, preferences, demographics

Multi-task	Relevance, diversity, fairness simultaneously
------------	---

Emerging research: Fairness & bias mitigation, explainability in ranking

3. Latent Semantic Indexing (LSI) — Extended Treatment

3.1 Mathematical Foundations

Concept	Definition
Matrix rank	Number of linearly independent rows/columns
Diagonal matrix	Non-zeros only on main diagonal
Eigenvalue λ	Scalar where $C^{**}x = \lambda x$ for non-zero x
Eigenvector x	Direction preserved by linear transformation C

3.2 The Vocabulary Mismatch Problem

Synonymy: Different words, same meaning (*car* $\approx$ *automobile*)

Polysemy: Same word, different meanings (*bank* = river bank vs. financial)

Example: Documents about "vehicle pollution" may use *exhaust gas*, *emissions*, *toxic fumes* — exact term matching fails.

3.3 SVD for LSI

Full SVD: For $m \times n$ matrix A (rank r): $A = U S V^T$

Where:

- U : $m \times r$ orthogonal (left singular vectors)
- S : $r \times r$ diagonal (singular values $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$)
- V : $n \times r$ orthogonal (right singular vectors)

Truncated SVD (rank $k \ll r$): $A_k = U_k S_k V_k^T = \sum_{i=1}^k \sigma_i u_i v_i^T$

Eckart-Young-Mirsky theorem: A_k is the optimal rank- k approximation minimizing $\|A - A_k\|_F$

3.4 Query Processing in LSI Space

Original query vector: $q \in \mathbb{R}^m$ (term space)

Project to k-dimensional concept space: $\tilde{q} = q^T U_k S_k^{-1}$

Document similarity: Compute cosine between $\tilde{q}$ and rows of $V_k S_k$ (document representations in concept space)

3.5 LSI Properties

Advantage	Limitation
Captures latent semantic relationships	Optimal k must be determined empirically
Reduces dimensionality (noise reduction)	Computationally expensive SVD for large collections
Handles synonymy and some polysemy	Interpretability of dimensions is limited
Improves recall over VSM	Static — no query-adaptive learning

4. Neural Embeddings for IR

4.1 Word Embeddings

Model	Core Idea	Key Property
Word2Vec (Mikolov et al.)	Predict context from word (CBOW) or word from context (Skip-gram)	Dense, semantic vectors
GloVe	Global word-word co-occurrence statistics	Captures global corpus statistics
FastText	Subword character n-grams	Handles out-of-vocabulary words

Semantic properties: *king* – *man* + *woman* $\approx$ *queen* (analogical reasoning)

4.2 Document/Sentence Embeddings

Approach	Method	Characteristics
Averaging	Mean of word vectors	Simple, fast, loses word order
TF-IDF weighting	Weighted average	Emphasizes discriminative terms
Smooth Inverse Frequency (SIF)	Weighted average with common component removal	Better sentence similarity
Doc2Vec (Paragraph Vector)	Extension of Word2Vec with document ID	Learns document-specific embeddings
Neural encoders	RNNs, CNNs, Transformers	Context-aware, order-sensitive

4.3 Embedding-Based Retrieval

Stage	Approach	Models
Dense retrieval	Encode query and docs to same space, use ANN	DPR, ANCE, Contriever

Re-ranking	Cross-encoder for fine-grained scoring	BERT, RoBERTa, ELECTRA
Hybrid	Sparse + dense combination	SPLADE, ColBERT

5. Transformers & BERT for IR 🚀

5.1 Self-Attention Mechanism

Attention: "The ability to focus on relevant parts of input when producing each output"

Scaled dot-product attention: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$

Where:

- Q (queries), K (keys), V (values) = linear projections of input
- d_k = dimension of key vectors (scaling factor prevents softmax saturation)

Multi-head attention: Parallel attention heads with different learned projections, concatenated and linearly transformed.

5.2 BERT Architecture

Component	Specification
Base	12 layers, 768 hidden, 12 heads, 110M parameters
Large	24 layers, 1024 hidden, 16 heads, 340M parameters
Training	Masked LM + Next Sentence Prediction
Input	[CLS] query [SEP] document [SEP]

5.3 BERT for IR — Two-Stage Paradigm

Stage	Architecture	Purpose	Efficiency
First stage: Retrieval	Bi-encoder (DPR, ANCE)	Fast candidate selection	$O(n)$ with ANN indexing
Second stage: Re-ranking	Cross-encoder (BERT)	Fine-grained relevance scoring	$O(k)$ for top-k candidates

Bi-encoder:

- Query and document encoded independently
- Similarity = dot product or cosine of [CLS] embeddings
- Enables approximate nearest neighbor (ANN) search

Cross-encoder:

- Query and document concatenated with [SEP]
- Full self-attention interaction
- More accurate but computationally expensive

5.4 Training Strategies for Neural IR

Approach	Method	Data
Supervised	Cross-entropy on relevance labels	MS MARCO, TREC collections
Contrastive	InfoNCE, margin-based losses	In-batch negatives, hard negatives
Distillation	Teacher-student, ColBERT compression	BERT → efficient student
Unsupervised	Inverse Cloze Task, ICT, ICT-like	Large unlabeled corpora

6. Large Language Models for IR 🤖

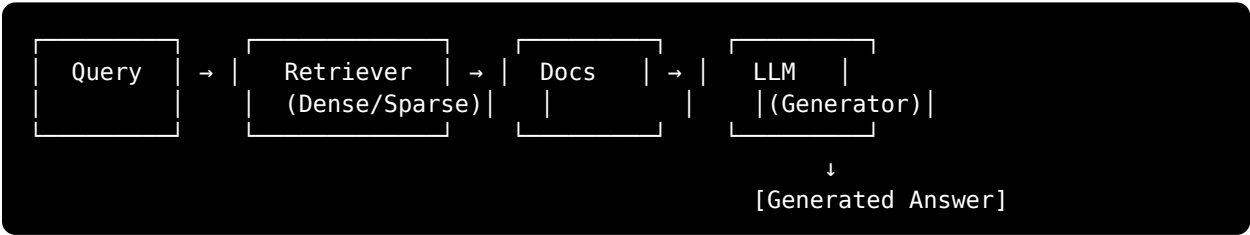
6.1 Capabilities & Opportunities

Capability	IR Application
Query understanding	Intent detection, query rewriting, expansion
Document understanding	Summarization, entity extraction, structure parsing
Generation	Query suggestion, answer generation, RAG
Reasoning	Multi-hop retrieval, evidence synthesis
Evaluation	LLM-as-judge for relevance assessment

6.2 Retrieval-Augmented Generation (RAG)

RAG: Augment LLM generation with retrieved relevant documents to ground responses in factual, up-to-date information.

Architecture:



Benefits:

- Reduces hallucination
- Enables real-time knowledge without retraining
- Provides source attribution

6.3 LLM-as-Judge

Application	Method
Relevance labeling	Prompt LLM to assess query-document relevance
Pairwise comparison	Which document is more relevant?
Listwise ranking	Rank this set by relevance
Explanation generation	Why is this relevant?

Challenges: Position bias, verbosity bias, self-preference, calibration to human judgments.

6.4 Autonomous Retrieval Agents

Emerging paradigm where LLMs:

- **Decompose** complex queries into sub-queries
- **Select** appropriate retrieval tools/APIs
- **Synthesize** multi-source information
- **Verify** and **iterate** based on intermediate results

7. Summary: Evolution of IR Ranking

Era	Paradigm	Key Innovation	Limitation
1960s–1990s	Boolean/VSM	Term matching, TF-IDF	Vocabulary mismatch
1990s–2000s	Probabilistic/LM	Statistical relevance, smoothing	Still lexical
1998–2000s	Link analysis	PageRank, HITS — authority signals	Query-independent
2000s–2010s	Learning to Rank	Data-driven feature combination	Feature engineering burden
2010s–2018	Neural embeddings	Dense semantic representations	Limited context, shallow
2018–2022	Transformers/BERT	Deep contextual understanding	Computational cost, two-stage needed
2022–present	LLMs for IR	Generative, reasoning, autonomous	Hallucination, latency, evaluation

Current frontier: Integrating LLM capabilities with efficient retrieval — RAG, LLM reranking, and autonomous retrieval agents represent the active research and deployment space.

Putting it All Together

Evaluation Foundations

Evaluation is the systematic process of measuring how well a retrieval system satisfies users' information needs.

- **Core metrics** – precision, recall, F-measure, MAP, GMAP, nDCG, reciprocal rank – capture different aspects of ranking quality.
- **Set-based vs. ranking-sensitive** – early metrics (precision/recall) ignore order; later metrics (AP, nDCG) reward early placement of relevant items.
- **Statistical testing** (e.g., paired t-test, bootstrap) ensures that observed differences between systems are not due to chance.

Metric	What it rewards	Typical use
Precision@k	Relevant docs in the top-k	Web search where users view only the first few results
R-Precision	Precision at the true number of relevant docs	Collections with known relevance counts
MAP	Average early relevance across queries	Benchmark competitions (TREC, FIRE)
GMAP	Penalises very poor performance on any query	Safety-critical retrieval tasks
nDCG	Graded relevance with logarithmic discount	E-commerce and news feeds

Benchmarking & Tooling

The **Cranfield method** provides a repeatable framework: a fixed document set, a fixed query set, and relevance judgments.

- **Standard collections** (TREC, CLEF, FIRE, NTCIR) supply the three components needed for reproducible experiments.
- **trec_eval** automates metric computation:

```
trec_eval -m map -m P.10 -m ndcg qrels.txt run.txt
```

Options (-m for selecting metrics, -q for per-query scores) make it easy to generate comprehensive reports.

- **Interpretation tips** – a MAP below 0.1 usually signals a broken pipeline; a P@10 of 0 means no relevant document appears in the first page.

Aspect	Traditional IR	Web-scale IR
Data source	Curated corpora	Billion-page crawls
Evaluation tool	trec_eval (or custom scripts)	trec_eval plus large-scale log-analysis
Typical metrics	MAP, precision/recall	nDCG@10, MRR, click-through-rate

Web-Scale Data Acquisition

A **web crawler** is an automated agent that discovers and downloads pages, building the raw material for any IR system.

- **Frontier management** (priority queue, politeness policies, revisit scheduling) balances **coverage** (how many pages are fetched) against **freshness** (how up-to-date the index is).
- **Link analysis** supplies query-independent quality signals:
 - **PageRank** computes a stationary distribution over the link graph, using the random-surfer model with damping factor ≈ 0.85 .
 - **HITS** extracts a **hub** score (pages that point to many authorities) and an **authority** score (pages cited by good hubs) from a query-specific subgraph.

Feature	PageRank	HITS
Computation	Global, pre-computed	Local, query-time
Score type	Single importance value	Hub & authority pair
Strength	Stable, captures overall popularity	Highlights topical communities
Weakness	Favors older pages, vulnerable to link spam	Sensitive to seed set, less scalable

- **Spam mitigation** – budget-enforced crawling, BEAST, duplicate detection (shingling + min-hash + LSH) keep the index clean.

From Classical to Neural Ranking

Learning to Rank (LTR) reframes ranking as a supervised learning problem, letting data dictate the optimal scoring function.

- **Feature families** – content scores (BM25, LM), static signals (PageRank, URL depth), interaction features (embedding similarity), and cross-modal cues.
- **Training paradigms** –
 - *Pointwise* (regression on individual pairs)
 - *Pairwise* (classification of document pairs; e.g., RankNet, LambdaRank)
 - *Listwise* (optimising the entire ranked list; e.g., ListNet, LambdaMART).
- **Neural LTR** couples deep encoders (BERT, Siamese networks) with pairwise or listwise losses, yielding **interaction-focused** models such as ColBERT that excel on dense retrieval tasks.
- **Latent Semantic Indexing (LSI)** reduces the term-document matrix via SVD, uncovering hidden concepts and alleviating vocabulary mismatch.
- **Embeddings & Transformers** map queries and documents into a shared dense space, enabling semantic matching beyond exact term overlap.
- **Large Language Models (LLMs)** extend this pipeline:
 - *Query understanding* (re-writing, expansion)
 - *RAG* (retrieval-augmented generation) where the model retrieves supporting passages before generating answers
 - *LLM-as-judge* for automatic relevance assessment in the evaluation loop.

Stage	Typical technique	Example
Initial retrieval	BM25, LM, LSI	Fast inverted-index lookup
First-pass ranking	LTR with hand-crafted features	LambdaMART on BM25 + PageRank
Re-ranking	Neural interaction models	BERT-based cross-encoder
Post-processing	Rerank with LLM-generated scores	GPT-4 re-ranker using relevance prompts

 How the Pieces Fit Together

- **Crawling** supplies the massive document collection; **link analysis** (PageRank/HITS) enriches each document with a static quality signal.
- **Benchmark datasets** (TREC, FIRE) provide the query set and relevance judgments that feed **trec_eval** and serve as the training/evaluation ground for **LTR** and **neural** models.
- **Evaluation metrics** close the loop: they quantify the impact of every design choice—from the choice of indexing pipeline to the loss function used in LTR.
- **Modern pipelines** often cascade: a fast lexical retriever (BM25/LSI) produces candidates, LTR (pointwise/pairwise/listwise) re-orders them using both content and link features, and a transformer-based re-ranker (or LLM) applies deep semantic reasoning before the final list is judged with nDCG or MAP.

By aligning **data acquisition**, **quality signals**, **learning frameworks**, and **rigorous evaluation**, a coherent end-to-end IR system emerges—one that can scale to the web while remaining empirically grounded.